# Guia Prático & Atividade de Laboratório — Aula 06

*Segurança em APIs REST: OAuth2 Resource Server, JWT (RS256) e Controlo de Acesso Baseado em Scopes no Spring Security*

---

## 1. Resumo Teórico & Guia de Estudo (Fundamentos de Segurança)

Nas Aulas 01 a 05, desenvolvemos a API MyMovies com persistência JPA, integração externa TMDB, tratamento de erros ApiError, testes unitários Mockito e observabilidade Actuator. Na **Aula 06**, implementamos a camada de **Segurança e Controlo de Acesso**, transformando a API num **OAuth2 Resource Server** capaz de validar tokens JWT (RS256) de forma totalmente Stateless.

### 1.1. Autenticação (AuthN) vs. Autorização (AuthZ)

É fundamental distinguir dois conceitos que regem a segurança de qualquer API corporativa:

- **Autenticação (AuthN - Quem és tu?):** Validação da identidade do utilizador. Se o token JWT estiver ausente, expirado ou com a assinatura digital corrompida, o Spring Security rejeita a requisição com o código HTTP **401 Unauthorized**
- **Autorização (AuthZ - O que podes fazer?):** Verificação das permissões associadas à identidade. Se o utilizador apresentar um token autêntico e válido, mas tentar aceder a um endpoint administrativo sem possuir o scope **movies.admin**, o Spring Security bloqueia o acesso com o código HTTP **403 Forbidden**

### 1.2. A Arquitetura Stateless do OAuth2 Resource Server

Em APIs REST modernas, a aplicação não mantém sessões HTTP em memória (**SessionCreationPolicy.STATELESS**). Cada requisição HTTP transporta de forma autónoma a sua credencial através do cabeçalho **Authorization: Bearer <token>**. O Resource Server valida a credencial sem consultar a base de dados a cada chamada.

### 1.3. Anatomia do JSON Web Token (JWT) e Assinatura Assimétrica (RS256)

Um token JWT é composto por três partes codificadas em Base64URL (**Header.Payload.Signature**):

- **Header:** Especifica o tipo de token (JWT) e o algoritmo de assinatura (RS256 - RSA Signature SHA-256).
- **Payload (Claims):** Contém os dados da sessão (sub / subject, iss / issuer, exp / expiration, e scope / autoridades).
- **Signature (Assinatura Assimétrica):** Garantia de integridade e autenticidade. O emissor assina o token com a sua **Chave Privada RSA**. A API MyMovies valida a assinatura utilizando exclusivamente a **Chave Pública RSA** (classroom-public.pem).

### 1.4. Mapeamento Automático de Scopes para Spring GrantedAuthorities

Por omissão, o Spring Security analisa o claim **"scope"** presente no Payload do JWT e converte cada elemento numa **GrantedAuthority** com o prefixo **SCOPE_**:

| Claim no JWT Payload | GrantedAuthority no Spring | Regra na SecurityFilterChain |
|---|---|---|
| "scope": "movies.read" | `SCOPE_movies.read` | `.hasAuthority("SCOPE_movies.read")` |
| "scope": "movies.admin" | `SCOPE_movies.admin` | `.hasAuthority("SCOPE_movies.admin")` |

### 1.5. Ponte Didática: Da API do TMDB (Aula 04) ao OAuth2 Resource

> **PONTE DIDÁTICA: INVERSÃO DE PAPÉIS (TMDB vs. MYMOVIES API)**
>
> Na Aula 04, a nossa aplicação MyMovies atuou como um CLIENTE HTTP que consumia dados da API externa do TMDB. Para obtermos permissão de consulta aos filmes, passámos no cabeçalho HTTP a nossa chave de leitura: Authorization: Bearer <TMDB_READ_ACCESS_TOKEN>. Naquela aula, o TMDB desempenhou o papel de Resource Server.
>
> Na Aula 06, ocorre o 'Lado Inverso do Espelho': a nossa API MyMovies assume exatamente a posição de RESOURCE SERVER (igual ao TMDB!). Agora é o Bruno Client (ou a aplicação frontend/mobile) que atua como Cliente e envia o token Bearer para aceder aos nossos endpoints. O padrão de autorização com Bearer Token é exatamente o mesmo nas duas pontas!

### 1.6. Arquitetura Real de Produção vs. Ambiente de Laboratório

É vital compreender a diferença entre o ambiente real corporativo e a simplificação didática adotada no nosso laboratório:

- **Como funciona em PRODUÇÃO (Identity Provider / Keycloak / Auth0):** A nossa API de filmes nunca gera, assina ou emite tokens JWT, nem armazena chaves em ficheiros locais. A responsabilidade de autenticação é 100% delegada a um Servidor de Autorização dedicado (Identity Provider - IdP). O utilizador faz login no IdP, obtém o JWT assinado e envia-o para a nossa API. A nossa API descarrega a chave pública do IdP via rede no arranque através da propriedade **spring.security.oauth2.resourceserver.jwt.jwk-set-uri** e valida os tokens recebidos de forma automática.
- **Como funciona no NOSSO LABORATÓRIO (Zero Infrastructure Friction):** Para manter o foco total na aprendizagem das regras do Spring Security e evitar a complexidade de subir um servidor Keycloak no computador de cada aluno, utilizamos tokens JWT pré-assinados pela chave RSA da turma. A chave pública **classroom-public.pem** é carregada localmente a partir de **classpath:keys/classroom-public.pem**. O mecanismo interno de validação criptográfica do Spring Security é 100% idêntico ao de produção!

## 2. Baseline Git & Instruções Didáticas de Preparação do Ambiente

Antes de iniciar a codificação, certifique-se de que o repositório local está atualizado a partir da branch oficial:

```
git checkout main
git pull origin main
git checkout -b feature/aula-06-spring-security checkpoint/aula-05-quality
```

### 2.1. Guia: Utilização do Bruno Client no Laboratório

Ao efetuar o **git pull**, a coleção do Bruno Client já inclui a chave pública em **src/main/resources/keys/classroom-public.pem** e o ambiente **Classroom-Local** pré-configurado com as variaveis que receberão os tokens de teste. Siga estes passos para testar:

- **Passo 1:** Abra o Bruno Client e abra a pasta da coleção do curso.
- **Passo 2:** No canto superior direito do Bruno, clique no menu suspenso de ambientes e selecione **local**.
- **Passo 3:** Observe as variáveis pré-carregadas que simulam as diferentes identidades da turma:

| Variável no Bruno | Identidade / Subject (sub) | Scopes & Comportamento Esperado |
|---|---|---|
| {{user_token}} | aluno@rumos.pt | movies.read ratings.write<br>(Acede a endpoints autenticados comuns. Retorna 403 em rotas admin). |
| {{admin_token}} | admin@rumos.pt | movies.read movies.admin<br>(Possui privilégios elevados. Acede a rotas administrativas /api/admin/** com 200 OK). |
| {{expired_token}} | aluno@rumos.pt | Validade no passado (exp)<br>(Simula token antigo. O Spring Security rejeita imediatamente com 401 Unauthorized). |

### 2.2. Estrutura Oficial de Pacotes do Projeto (Aula 06)

```
pt.rumos.mymovies
├── config
│   └── SecurityConfig.java          <-- NEW: Configuração do OAuth2 Resource Server & JWT
├── controller
│   ├── MovieController.java         <-- UPDATE: REST Controller – Rating protegido
│   └── AdminController.java         <-- NEW: REST Controller Restrito a Admin (/api/admin)
src/main/resources
├── keys
│   └── classroom-public.pem         <-- NEW: Chave Pública RSA (RS256)
└── application.properties           <-- Atualizado com localização da chave pública
```

## 3. Roteiro Guiado da Atividade Prática (Passo a Passo)

### Etapa 1: Adicionar o Starter do OAuth2 Resource Server (pom.xml)

Abra o ficheiro **pom.xml** e adicione a dependência oficial do Spring Security OAuth2 Resource Server:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security-oauth2-resource-server</artifactId>
</dependency>
```

### Etapa 2: Configurar a Chave Pública no application.properties

No ficheiro **src/main/resources/application.properties**, adicione o caminho para a chave pública RSA:

```
# Spring Security - OAuth2 Resource Server JWT Configuration
jwt.public.key=classpath:keys/classroom-public.pem
spring.security.oauth2.resourceserver.jwt.public-key-location=${jwt.public.key}
```

### Etapa 3: Criar a Classe de Configuração SecurityConfig.java

Crie o pacote **pt.rumos.mymovies.config** e adicione a classe **SecurityConfig.java**. Configure a **SecurityFilterChain** com política de sessão Stateless e regras de autorização por URL:

```
package pt.rumos.mymovies.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.oauth2.jwt.JwtDecoder;
import org.springframework.security.oauth2.jwt.NimbusJwtDecoder;
import org.springframework.security.web.SecurityFilterChain;

import java.security.interfaces.RSAPublicKey;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Value("${jwt.public.key}")
    private RSAPublicKey publicKey;
```

```
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
                .csrf(csrf -> csrf.disable())
                .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers(HttpMethod.GET, "/api/movies/**").permitAll()
                        .requestMatchers("/actuator/health").permitAll()
                        .requestMatchers("/actuator/metrics/**").permitAll()
                        .requestMatchers("/h2-console/**").permitAll()

.requestMatchers("/api/admin/**").hasAuthority("SCOPE_movies.admin")
                        .anyRequest().authenticated()
                )
                .oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults()))
                .headers(headers -> headers.frameOptions(frame -> frame.sameOrigin()))
                .build();
    }

    //Aqui o Bean abaixo é redundante, pois uma vez configurado no application.properties
    //e se não farei nenhuma configuração adicional, o Spring já inicializa e gerencia
    //o bean de decodificação. Mantido aqui para visualizar onde fica em caso de ausência
    //no properties (oauth2.jwt(Customizer.withDefaults() – usa o bean gerado pelo Spring)
    @Bean
    public JwtDecoder jwtDecoder() {
        return NimbusJwtDecoder.withPublicKey(this.publicKey).build();
    }
}
```

## Etapa 4: Extrair a Identidade do Utilizador com @AuthenticationPrincipal

No método do controller protegido (ex.: **updateRating**), utilize a anotação **@AuthenticationPrincipal Jwt jwt** para extrair diretamente o subject do utilizador autenticado sem consultas adicionais (use o import *org.springframework.security.oauth2.jwt.Jwt*) :

```
@PatchMapping("/{id}/rating")
public ResponseEntity<MovieResponse> updateRating(@PathVariable Long id, @Valid
@RequestBody RatingUpdateRequest request, @AuthenticationPrincipal Jwt jwt) {
    String userEmail = jwt.getSubject(); // Extrai o "sub" do JWT (ex: aluno@rumos.pt)
    log.info("Registo de avaliação recebido do utilizador: {} para o filme ID: {}",
userEmail, id);

    var optionalMovieResponse = movieService.updateRating(id,request);

    if (optionalMovieResponse.isEmpty()) {
        return ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(optionalMovieResponse.get());
}
```

# 4. ATIVIDADE EXTRA & BATERIA DE TESTES NO BRUNO CLIENT

## Atividade Extra A: Implementar o AdminController.java

Crie o controller **AdminController.java** sob a rota **/api/admin/stats** para validar a restrição de acesso por scope administrativo:

```
package pt.rumos.mymovies.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.security.oauth2.jwt.Jwt;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.Map;

@RestController
@RequestMapping("/api/admin")
public class AdminController {

    @GetMapping("/stats")
    public ResponseEntity<Map<String, Object>> getSystemStats(@AuthenticationPrincipal Jwt
jwt) {
        return ResponseEntity.ok(Map.of(
                "status", "OPERATIONAL",
                "requestedBy", jwt.getSubject(),
                "totalMovies", 150,
                "activeUsers", 42
        ));
    }
}
```

## Atividade Extra B: Homologação de Respostas HTTP no Bruno Client

Execute os 5 cenários de teste no Bruno Client e confirme o código de resposta retornado pela MyMovies API:

| Cenário / Requisição | Auth Header / Token | Status HTTP Esperado | Motivo da Resposta |
|---|---|---|---|
| GET /api/movies/1 | Sem Token | **200 OK** | Rota pública permitida por permitAll(). |
| PATCH /api/movies/1/rating | {{user_token}} | **201 Created** | Token autêntico e válido de utilizador. |
| PATCH /api/movies/1/rating | {{expired_token}} | **401 Unauthorized** | Token rejeitado por data de expiração (exp) no passado. |

| | | | |
|---|---|---|---|
| GET /api/admin/stats | {{user_token}} | **403 Forbidden** | Token autêntico, mas sem o scope SCOPE_movies.admin. |
| GET /api/admin/stats | {{admin_token}} | **200 OK** | Token com scope SCOPE_movies.admin autorizado. |

## 5. Checklist de Conclusão

Antes de finalizar a aula e efetuar a entrega, confirme todos os pontos da Definition of Done:

- **[ ]** O starter **spring-boot-starter-security-oauth2-resource-server** está adicionado ao pom.xml.
- **[ ]** A chave pública **classroom-public.pem** está presente em **src/main/resources/keys/** e configurada no application.properties.
- **[ ]** Configuração do JwtDecoder no **application.properties**
- **[ ]** A **SecurityFilterChain** bloqueia acessos a **/api/admin/**** sem o scope **SCOPE_movies.admin** (HTTP 403).
- **[ ]** Tokens expirados ou ausentes em rotas protegidas retornam **HTTP 401 Unauthorized**.
- **[ ]** Os 5 testes no Bruno Client com o ambiente **local** responderam com os códigos HTTP previstos.